Universidad Autónoma de Tamaulipas
Unidad Académica Multidisciplinaria Mante Centro

Nombre del trabajo:
Reporte práctica 8

Asignatura:
Arquitectura de Equipos Digitales

Profesor:
López piña Daniel

Alumno:
Guerrero Mendoza Emmanuel

Grado: 6° **Grupo:** EF

Ciudad Mante, Tamaulipas Abril 2026

Introducción

En los sistemas embebidos, la visualización de información es fundamental para la interacción con el usuario. Las pantallas TFT permiten mostrar datos de forma gráfica y en tiempo real, mejorando la presentación y comprensión de la información.

En esta práctica se utilizó una pantalla TFT con controlador ILI9341, comunicada mediante el protocolo SPI. Se desarrolló una interfaz gráfica que simula una estación ambiental, mostrando datos como temperatura y humedad de forma dinámica.

Desarrollo

Para la implementación se utilizaron las librerías:

```
#include <Adafruit_GFX.h>
#include <Adafruit_ILI9341.h>
#include <SPI.h>
```

Se definieron los pines de conexión SPI para el microcontrolador (ESP32):

CS (Chip Select) → Pin 5
DC (Data/Command) → Pin 21
RST (Reset) → Pin 22

En la función setup(), se inicializó la comunicación serial y la pantalla TFT mediante tft.begin(). Posteriormente, se configuró la orientación de la pantalla en modo horizontal (setRotation(1)).

Se mostró una pantalla de bienvenida con texto informativo y, después de un breve tiempo, se limpió la pantalla para dibujar la interfaz principal.

Diseño de la interfaz

Se implementó una función dibujarPanel() que construye la interfaz gráfica, la cual incluye:

- Una barra superior con título del sistema
- Un recuadro para mostrar la temperatura

- Un recuadro para mostrar la humedad
- Un área inferior destinada a historial o gráfica

Se utilizaron funciones como:

- fillRect() para dibujar áreas sólidas
- drawRoundRect() para marcos
- setCursor() y print() para texto

Actualización de datos

En la función loop(), se simulan valores de temperatura y humedad utilizando funciones matemáticas (sin y cos) para generar variaciones dinámicas.

Los datos se actualizan cada 500 ms mediante la función actualizarDatos(), la cual:

- Borra el área anterior con fillRect()
- Escribe nuevos valores con print()
- Mantiene la interfaz limpia sin redibujar toda la pantalla

Resultados

El sistema logró mostrar correctamente la información en la pantalla TFT. Se observó que:

- La interfaz gráfica se dibuja correctamente
- Los valores se actualizan en tiempo real
- La pantalla responde de forma fluida
- El uso de SPI permite una comunicación rápida

PREGUNTAS DE REFLEXION

•Cuántos bytes de datos se necesitan para actualizar un solo pixel en la pantalla ILI9341? ¿Y para actualizar toda la pantalla (240x320)?

R= Por pixel: 2 bytes (16 bits, formato RGB565)
Pantalla completa (240×320):
240 × 320 × 2 = 153,600 bytes

• A 40 MHz de SPI, ¿cuánto tiempo mínimo (teórico) se necesita para enviar todos los Datos de un frame completo? ¿Por qué en la práctica es mayor?

R=Tiempo teórico:
153,600 bytes × 8 bits = 1,228,800 bits
1,228,800 / 40,000,000 ≈ 0.0307 s (~30.7 ms)
En la práctica es mayor por:
Overhead, comandos, latencias y procesamiento

• ¿Por qué se usa la función fillRect() para borrar el área del número antes de redibujar El nuevo valor, en lugar de usar fillScreen() completo?

R= Para actualizar solo una región específica, Evita redibujar toda la pantalla, Mejora rendimiento y velocidad

• Compara la velocidad de actualización de la pantalla cuando el ESP32 opera el SPI a 10 MHz vs 40 MHz. ¿Qué diferencia se observa visualmente?

R=A 10 MHz: actualización más lenta, visible lag
A 40 MHz: actualización más fluida y rápida

• ¿Cuáles son las ventajas de usar SPI sobre I2C para una pantalla grafica de alta Resolución? ¿Existiría alguna ventaja de usar I2C para este caso?

R= Ventajas SPI:
Mayor velocidad
Mejor para gráficos
Mayor ancho de banda

I2C:
Más lento
Solo ventaja: menos cables

Conclusión

La práctica permitió comprender el uso del protocolo SPI y la implementación de interfaces gráficas en sistemas embebidos. Se logró diseñar una interfaz visual funcional capaz de mostrar datos dinámicos.

Esto demuestra la importancia de la visualización en proyectos electrónicos, facilitando la interacción entre el usuario y el sistema.

Bibliografía

- Arduino. (s.f.). Arduino Documentation. Recuperado de https://www.arduino.cc
- Adafruit. (s.f.). ILI9341 TFT Display Library.
- Banzi, M., & Shiloh, M. (2014). Getting Started with Arduino. Maker Media.
- Monk, S. (2017). Programming Arduino. McGraw-Hill Education.